



Python Dictionaries

Data with Dylan | Cheat Sheet | Python Fundamentals #5

WHAT IS A DICTIONARY?

Stores data in key-value pairs: use a key to look up its matching value.

```
student = {  
    "name": "Dylan",  
    "grade": 95  
}
```

◆ Real-world dictionary: word = key, definition = value.

CORE DICTIONARY RULES

RULE	MEANING	EXAMPLE
Key-value	key maps to value	"Alice": 90
Mutable	add/update/delete	grades["Bob"] = 88
Ordered	keeps insert order	dict prints in order
Unique keys	last duplicate wins	{"fruit": "grapes"}
Immutable keys	str/int/tuple OK	list key = TypeError

KEY SYNTAX & OPERATIONS

Create, access, add/update, delete:

```
grades = {"Alice": 90, "Bob": 85}  
grades["Alice"] # 90  
grades.get("Eve", 0) # 0, no error  
grades["Diana"] = 92 # add new pair  
grades["Alice"] = 95 # update value  
del grades["Bob"] # delete pair  
grades.pop("Eve", None) # safe delete
```

TYPE / FORMATTING WARNINGS

- ▲ Missing key with [] raises `KeyError`; use `.get()`.
Safer code: `grades.get("Eve", 0)`
- ▲ Keys must be immutable; lists/sets cannot be keys.
Valid example: `{"Alice", 1}: 90`
- ▲ Duplicate keys do not store both values.
Meaning: newest value wins.

🔪 MINI PRACTICE

```
profile = {"name": "Dylan", "points": 10, "level": 1}
```

Task: add 5 points, set level to 2, safely read "email".

Hint: use `["points"] += 5`, `["level"] = 2`, `.get("email", "missing")`

COMMON DICTIONARY METHODS

METHOD	WHAT IT DOES	CODE EXAMPLE
<code>.get()</code>	safe lookup with default	<code>grades.get("Ana", 0)</code>
<code>.keys()</code>	all dictionary keys	<code>grades.keys()</code>
<code>.values()</code>	all dictionary values	<code>grades.values()</code>
<code>.items()</code>	key/value pairs as tuples	<code>grades.items()</code>
<code>.update()</code>	merge or update pairs	<code>grades.update({"Ana": 97})</code>
<code>.pop()</code>	remove key + return value	<code>grades.pop("Bob", None)</code>

LOOPING PATTERNS

Use keys for lookup; use `.items()` for key + value:

```
for name in grades:  
    print(name, grades[name])  
  
for name, score in grades.items():  
    if score > 85:  
        print(name, score)
```

COMMON BEGINNER MISTAKES

- ✗ Missing key causes an error
`grades["Eve"]`
✓ `grades.get("Eve", 0)`
- ✗ Using a list as a key
`{["Alice"]: 90}`
✓ `{("Alice", "quiz1"): 90}`
- ✗ Duplicate keys overwrite earlier values
`{"fruit": "apple", "fruit": "grapes"}`
✓ `{"fruit_1": "apple", "fruit_2": "grapes"}`
- ✗ Comma instead of colon
`{"Alice", 90}`
✓ `{"Alice": 90}`